

CSE 101 Quiz 4 Review

Binary Search Trees, Minimum Spanning Trees, and Shortest Paths

Related Links:

- BST visualization: <https://visualgo.net/en/bst>
- Graph visualization: <https://visualgo.net/en/dfsdfs>
- MST visualization: <https://visualgo.net/en/mst>
- Dijkstra visualization: <https://visualgo.net/en/sssp>
- Kruskal explanation: <https://opensa-server.cs.vt.edu/ODSA/Books/Everything/html/Kruskal.html>
- Prim explanation: <https://opensa-server.cs.vt.edu/ODSA/Books/Everything/html/Prim.html>
- Dijkstra explanation: <https://opensa-server.cs.vt.edu/ODSA/Books/Everything/html/Dijkstra.html>

1. Binary Search Tree Review: Using Bounds

A Binary Search Tree, or BST, follows the ordering rule:

None

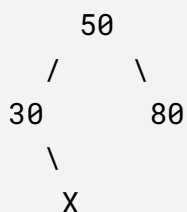
left subtree keys < node key < right subtree keys

When values are unique, no two nodes should contain the same number.

Instead of checking only a node's direct parent, it is often safer to think in terms of allowed ranges.

Example:

None



Since X is in the left subtree of 50, X must be less than 50.

Since X is in the right subtree of 30, X must be greater than 30.

So:

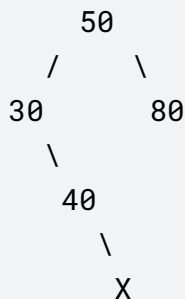
None

$$30 < X < 50$$

Any valid unique number in that range could fit, unless more surrounding nodes create more restrictions.

More detailed example:

None



X is:

- in the left subtree of 50, so $X < 50$
- in the right subtree of 30, so $X > 30$
- in the right subtree of 40, so $X > 40$

So the strongest condition is:

None

$$40 < X < 50$$

Common Mistakes:

- Only comparing X with its parent.
- Forgetting that ancestors also restrict the value.
- Placing a value correctly relative to the parent but incorrectly relative to the root.
- Forgetting that unique values means X cannot equal an existing node.

2. BST Traversal Reminder

The main depth-first traversals are:

Preorder:

None

Root, Left, Right

Inorder:

None

Left, Root, Right

Postorder:

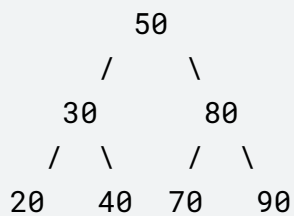
None

Left, Right, Root

For this review, focus on tracing the tree correctly, not just memorizing the names.

Example tree:

None



Preorder:

None

50 30 20 40 80 70 90

Explanation:

Visit 50 first, then traverse the whole left subtree, then traverse the whole right subtree.

Postorder:

None

20 40 30 70 90 80 50

Explanation:

Traverse the left subtree, then the right subtree, then visit the root last.

Inorder:

None

20 30 40 50 70 80 90

Important BST fact:

For a BST, inorder traversal gives values in sorted order.

Common Mistakes:

- Writing the root too early in postorder.
- Writing the root too late in preorder.
- Traversing only one level instead of recursively traversing each subtree.
- Forgetting that traversal means visiting every node exactly once.

3. How to Trace Preorder Carefully

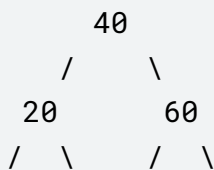
Preorder is:

None

Root, Left, Right

Use this tree:

None



```

10  30  50  70
    \
    35

```

Step-by-step preorder:

Start at root:

None
40

Go left to 20:

None
40 20

Go left to 10:

None
40 20 10

10 has no children, return to 20.

Go right to 30:

None
40 20 10 30

30 has no left child, go right to 35:

None
40 20 10 30 35

Left subtree is done. Now go right from 40 to 60:

None

40 20 10 30 35 60

Then 50, then 70:

None

40 20 10 30 35 60 50 70

Final preorder:

None

40 20 10 30 35 60 50 70

A good way to avoid mistakes:

When using preorder, write a node down the first time you arrive at it.

4. How to Trace Postorder Carefully

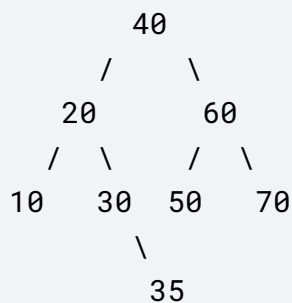
Postorder is:

None

Left, Right, Root

Use the same tree:

None



Start at 40, but do not write 40 yet.

Go to the left subtree rooted at 20.

For node 20:

- visit left subtree: 10
- visit right subtree: 30 and 35
- then visit 20

Postorder of left subtree:

None
10 35 30 20

Now go to the right subtree rooted at 60.

Postorder of right subtree:

None
50 70 60

Finally visit root 40.

Final postorder:

None
10 35 30 20 50 70 60 40

A good way to avoid mistakes:

When using postorder, write a node only when you are leaving it for the last time.

5. BST Insertion

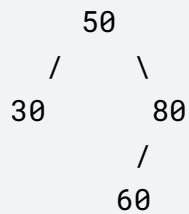
To insert a value into a BST:

1. Start at the root.
2. If the new value is smaller, go left.
3. If the new value is larger, go right.
4. Repeat until you reach an empty child position.
5. Insert the new value there.

Example: Insert 55.

Original tree:

None



Steps:

None

55 > 50, go right to 80

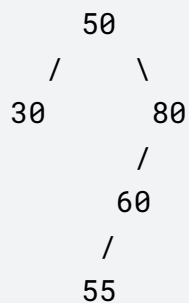
55 < 80, go left to 60

55 < 60, go left

left child is empty, insert 55 there

After insertion:

None



Important:

A regular BST insertion does not automatically rebalance the tree.

It only follows the BST ordering rule.

Common Mistakes:

- Inserting the value based only on the root.
- Forgetting to keep comparing as you move down.

- Putting the new node between two existing nodes instead of attaching it at an empty child.
- Accidentally changing the shape of unrelated parts of the tree.

6. Graph Terminology Needed for MST and Shortest Paths

A graph has:

- vertices, also called nodes
- edges, which connect vertices
- weights, which are costs assigned to edges

Example:

```
None
A --4-- B
|      /
2      1
|      /
C
```

Vertices:

```
None
A, B, C
```

Edges:

```
None
A-B with weight 4
A-C with weight 2
B-C with weight 1
```

For weighted graph problems, always pay attention to edge weights.

A lower edge weight means a cheaper connection, but the cheapest individual edge is not always part of every final answer.

7. Spanning Trees

A spanning tree is a set of edges that:

- includes all vertices
- keeps the graph connected
- has no cycles

For a connected graph with n vertices, every spanning tree has exactly:

None

$n - 1$ edges

Example:

If a graph has 6 vertices, a spanning tree must have:

None

$6 - 1 = 5$ edges

If you choose 6 edges for 6 vertices, there must be a cycle.

If you choose fewer than 5 edges for 6 vertices, the graph cannot be fully connected.

Common Mistakes:

- Forgetting that a spanning tree must include every vertex.
- Adding too many edges and creating a cycle.
- Stopping before all vertices are connected.
- Thinking a tree can have multiple disconnected components.

8. Minimum Spanning Tree

A Minimum Spanning Tree, or MST, is a spanning tree with minimum total edge weight.

MST problems usually apply to:

- connected graphs
- undirected graphs
- weighted graphs

The total cost is the sum of the selected edge weights.

Example selected MST edges:

None

A-B: 2

B-C: 3

C-D: 4

Total MST cost:

None

$2 + 3 + 4 = 9$

Important:

There can be more than one valid MST if edge weights create ties.

If a tie-breaking rule is given, follow it consistently.

Common Mistakes:

- Finding a shortest path instead of a spanning tree.
- Forgetting to include all vertices.
- Adding an edge that creates a cycle.
- Calculating the total cost using edges that were not selected.

9. Kruskal's Algorithm

Kruskal's algorithm builds an MST by considering edges from smallest weight to largest weight.

Main idea:

None

Sort all edges by weight.

Add the next smallest edge if it does not create a cycle.

Stop when all vertices are connected.

More formal steps:

1. List all edges with weights.
2. Sort edges from smallest to largest weight.
3. Start with no selected edges.
4. Consider edges one at a time in sorted order.

5. If the edge connects two different components, add it.
6. If the edge creates a cycle, skip it.
7. Stop after selecting $n - 1$ edges.

Example graph:

None

```

A --1-- B
| \    |
4  3    2
|      \ |
C --5-- D

```

Edges sorted by weight:

None

```

A-B: 1
B-D: 2
A-D: 3
A-C: 4
C-D: 5

```

Run Kruskal:

Pick A-B:

None

```

Selected: A-B
Cost: 1

```

Pick B-D:

None

```

Selected: A-B, B-D
Cost: 1 + 2 = 3

```

Consider A-D:

A-D would connect A to D, but A is already connected to D through A-B-D.

So adding A-D creates a cycle.

Skip A-D.

Pick A-C:

None

Selected: A-B, B-D, A-C

Cost: $1 + 2 + 4 = 7$

There are 4 vertices, so the MST needs 3 edges.

Stop.

Final MST edges:

None

A-B, B-D, A-C

Total cost:

None

7

10. Cycle Detection in Kruskal's Algorithm

The most important part of Kruskal's algorithm is avoiding cycles.

A simple way to think about this:

Before adding an edge, ask:

None

Are the two endpoints already connected through selected edges?

If yes, adding the edge creates a cycle.

If no, adding the edge is safe.

Example:

Selected edges:

None
A-B
B-C

This means A, B, and C are already connected.

Now consider edge A-C.

Adding A-C would create:

None
A-B-C-A

That is a cycle.

So skip A-C.

In implementation, this is often handled using Union-Find / Disjoint Set.

Conceptual Union-Find idea:

- Find tells which component a node belongs to.
- Union merges two components.
- If two nodes are already in the same component, do not add that edge.

Common Mistakes:

- Adding every low-weight edge without checking cycles.
- Thinking a cycle is only a triangle. Larger cycles also count.
- Forgetting to stop after $n - 1$ selected edges.
- Skipping an edge just because both endpoints already have some edge. The real question is whether the endpoints are already connected to each other.

11. Kruskal Tie-Breaking

If two edges have the same weight, a tie-breaking rule may be used.

Example tie rule:

None

Break ties alphabetically.

Suppose these edges have the same weight:

None

A-D: 3

B-C: 3

A-C: 3

Alphabetical order by edge name could be:

None

A-C

A-D

B-C

The exact expected ordering depends on how edges are named in the problem.

A safe approach:

1. Write all candidate edges clearly.
2. Sort first by weight.
3. For equal weights, apply the stated tie rule.
4. Still check whether each edge creates a cycle.

Important:

Tie-breaking affects the order of consideration.

It does not remove the need to check cycles.

Common Mistakes:

- Ignoring tie rules.
- Applying tie rules before sorting by weight.
- Forgetting that tied edges can still be skipped if they create cycles.

12. Prim's Algorithm

Prim's algorithm also builds an MST, but it grows one connected tree from a starting vertex.

Main idea:

None

Start from one vertex.

Repeatedly add the cheapest edge that connects the current tree to a new vertex.

Steps:

1. Start with the given starting node.
2. Mark it as part of the tree.
3. Look at all edges crossing from the tree to vertices outside the tree.
4. Pick the smallest crossing edge.
5. Add that edge and the new vertex.
6. Repeat until all vertices are included.

Important:

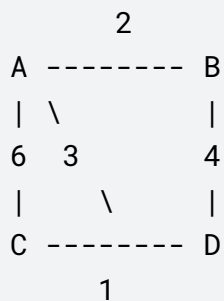
Prim's algorithm only considers edges that connect the current tree to an outside vertex.

It does not simply pick the globally smallest edge from the entire graph each time.

13. Prim's Algorithm Example

Graph:

None



Start at A.

Current tree:

None

A

Available crossing edges from A:

None

A-B: 2

A-C: 6

A-D: 3

Pick A-B because weight 2 is smallest.

Current tree:

None

A, B

Selected edges:

None

A-B

Now crossing edges from {A, B} to outside vertices {C, D}:

None

A-C: 6

A-D: 3

B-D: 4

Pick A-D because weight 3 is smallest.

Current tree:

None

A, B, D

Selected edges:

None

A-B, A-D

Now only C is outside.

Crossing edges to C:

None

A-C: 6

C-D: 1

Pick C-D because weight 1 is smallest.

Final selected edges:

None

A-B, A-D, C-D

Total cost:

None

$2 + 3 + 1 = 6$

Common Mistakes:

- Picking the globally smallest edge even if it does not touch the current tree.
- Adding an edge between two vertices that are both already inside the tree.
- Forgetting the starting vertex.
- Not updating the available crossing edges after adding a new vertex.

14. Prim Tie-Breaking

If two candidate edges have the same weight, use the tie-breaking rule given by the problem.

Example:

Current tree:

None

A, B

Outside vertices:

None

C, D

Candidate crossing edges:

None

A-C: 4

B-D: 4

A-D: 6

If ties are broken alphabetically, compare the tied choices:

None

A-C

B-D

Pick A-C first.

Important:

Tie-breaking is only among currently valid candidate edges.

Do not compare against edges that do not cross from the tree to an outside vertex.

Common Mistakes:

- Applying tie-breaking to every graph edge instead of only candidate crossing edges.
- Forgetting that Prim changes its candidate set every step.
- Choosing an edge that forms a cycle inside the current tree.

15. Kruskal vs Prim

Both Kruskal and Prim find a Minimum Spanning Tree.

But they build it differently.

Kruskal:

None

Sort all edges globally.

Add smallest safe edge.

Avoid cycles.

May grow multiple components before they merge.

Prim:

None

Start at one vertex.

Grow one connected tree.

Each step adds the cheapest edge from the tree to a new vertex.

Comparison:

Feature	Kruskal	Prim
Starts with	Empty forest	One start vertex
Main choice	Globally smallest safe edge	Smallest crossing edge
Cycle handling	Skip edges that create cycles	Avoid edges to already-in-tree vertices
Uses start node?	No	Yes
Intermediate structure	Multiple components possible	One connected tree

Important:

The final MST cost should be the same if the graph has a unique MST.

If there are ties, Kruskal and Prim may produce different MSTs with the same minimum total cost.

Common Mistakes:

- Running Prim like Kruskal.
- Running Kruskal like Prim.
- Thinking Prim's result must list edges in globally sorted order.
- Thinking the start node matters for Kruskal. It does not.

16. Shortest Path vs Minimum Spanning Tree

MST and shortest path problems are different.

Minimum Spanning Tree:

None

Connect all vertices with minimum total edge cost.

Shortest Path:

None

Find the cheapest path from a source vertex to other vertices.

Example:

None

```
A --1-- B --1-- C
 \           /
  \---3-----/
```

MST:

Choose:

None

A-B: 1
B-C: 1

Total cost:

None

2

Shortest path from A to C:

None

A-B-C

Cost:

None

2

In this example, they happen to match for A to C.

But they do not always match.

Different goal:

- MST cares about connecting the entire graph cheaply.
- Dijkstra cares about shortest distances from one source.

Common Mistakes:

- Using MST edges to answer shortest path questions without checking distances.
- Thinking the cheapest spanning tree gives all shortest paths.
- Thinking Dijkstra is used to find MSTs.

17. Dijkstra's Algorithm

Dijkstra's algorithm finds shortest paths from a starting source node to other nodes in a graph with nonnegative edge weights.

Main idea:

None

Keep the best known distance to each vertex.

Repeatedly finalize the unvisited vertex with the smallest known distance.

Relax its outgoing edges.

Key vocabulary:

Distance estimate:

None

Current best known cost from the source to a vertex.

Previous node:

None

The node used to reach this vertex on the current best path.

Visited / finalized:

None

The shortest distance to that node is finalized.

Relaxation:

None

Check whether going through the current node gives a shorter path.

Relaxation formula:

None

```
if dist[u] + weight(u, v) < dist[v]:  
    dist[v] = dist[u] + weight(u, v)  
    prev[v] = u
```

18. Dijkstra's Algorithm Table Setup

A clean Dijkstra table usually tracks:

Node	Known Distance	Previous Node	Finalized?
A	0	-	Yes/No
B	∞	-	Yes/No
C	∞	-	Yes/No

At the start:

- source distance = 0
- all other distances = ∞
- previous nodes are blank or undefined
- no nodes are finalized yet

Example if source is A:

Node	Distance	Previous
A	0	-
B	∞	-
C	∞	-
D	∞	-

Then start by processing A.

Common Mistakes:

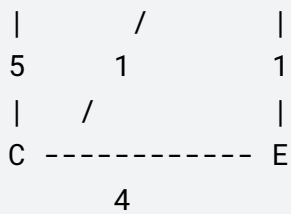
- Forgetting to initialize the source distance to 0.
- Forgetting to initialize other distances to infinity.
- Updating a distance but not updating the previous node.
- Finalizing a node too early.

19. Dijkstra's Algorithm Example

Graph:

None

A --2-- B --2-- D



Edges:

None

A-B: 2

A-C: 5

B-C: 1

B-D: 2

D-E: 1

C-E: 4

Start at A.

Initial distances:

None

A = 0

B = ∞

C = ∞

D = ∞

E = ∞

Process A:

Neighbors:

None

B through A: $0 + 2 = 2$

C through A: $0 + 5 = 5$

Update:

None

$B = 2, \text{prev}[B] = A$

$C = 5, \text{prev}[C] = A$

Now choose the unvisited node with smallest distance:

None

$B = 2$

Process B:

Check C through B:

None

$\text{dist}[B] + \text{weight}(B, C) = 2 + 1 = 3$

This is better than current $C = 5$.

Update:

None

$C = 3, \text{prev}[C] = B$

Check D through B:

None

$2 + 2 = 4$

Update:

None

$D = 4, \text{prev}[D] = B$

Next smallest unvisited node:

None

$$C = 3$$

Process C:

Check E through C:

None

$$3 + 4 = 7$$

Update:

None

$$E = 7, \text{ prev}[E] = C$$

Next smallest unvisited node:

None

$$D = 4$$

Process D:

Check E through D:

None

$$4 + 1 = 5$$

This is better than current $E = 7$.

Update:

None

$$E = 5, \text{ prev}[E] = D$$

Next node:

None

E = 5

Final distances:

None

A = 0

B = 2

C = 3

D = 4

E = 5

Previous nodes:

None

B <- A

C <- B

D <- B

E <- D

Shortest path from A to E:

Follow previous pointers backward:

None

E <- D <- B <- A

Reverse it:

None

A -> B -> D -> E

Cost:

None

5

20. Shortest Path Tree

A shortest path tree shows how each finalized vertex was reached from the source.

Use the previous node pointers from Dijkstra.

From the example above:

None

`prev[B] = A`

`prev[C] = B`

`prev[D] = B`

`prev[E] = D`

Shortest path tree edges:

None

A-B

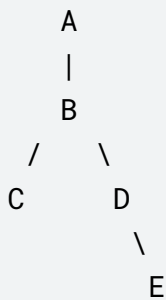
B-C

B-D

D-E

Drawn as a tree from source A:

None



Important:

The shortest path tree is not necessarily the same as the MST.

It is based on shortest paths from one source.

Common Mistakes:

- Drawing all relaxed edges instead of only final previous-pointer edges.
- Drawing the original graph again instead of the shortest path tree.
- Forgetting that each non-source node should have one previous node in the tree.
- Confusing shortest path tree cost with MST total cost.

21. Dijkstra Tie-Breaking

If two unvisited nodes have the same distance estimate, a tie-breaking rule may be used.

Example:

None

B = 4

C = 4

D = 7

If ties are broken alphabetically, choose B before C.

Tie-breaking can affect:

- the order nodes are finalized
- which previous pointer is kept when equal-cost paths exist
- the shape of the shortest path tree

Important:

If a new path has the same cost as the current best path, only update the previous pointer if the problem's tie rule says to do so.

For most hand-tracing problems, use the stated rule consistently.

Common Mistakes:

- Ignoring ties.
- Updating previous pointers randomly.
- Choosing alphabetically before comparing distance.
- Forgetting that smallest distance is still the first priority.

22. Recovering a Shortest Path

After Dijkstra finishes, the shortest path to a target is recovered using previous pointers.

Example previous pointers:

None

prev[B] = A

prev[C] = B

prev[D] = B

prev[E] = D

Find path from A to E:

Start at E:

None

E

Previous of E is D:

None

D -> E

Previous of D is B:

None

B -> D -> E

Previous of B is A:

None

A -> B -> D -> E

So the shortest path is:

None

A -> B -> D -> E

The cost is the final distance stored for E.

Common Mistakes:

- Listing previous pointers backward and forgetting to reverse the path.
- Adding edge weights from the wrong path.
- Reporting the previous node instead of the full path.
- Reporting the cost without showing how it was found.

23. Dijkstra vs BFS

BFS can find shortest paths in an unweighted graph.

Dijkstra is used when edges have weights.

If every edge has the same weight, BFS works because each edge contributes the same cost.

If edges have different weights, BFS is not enough because fewer edges does not always mean lower cost.

Example:

None

A --10-- B

A --1--- C --1-- B

BFS from A might reach B directly in one edge:

None

A -> B

But cost is:

None

10

The cheaper weighted path is:

None

A -> C -> B

Cost:

None

$$1 + 1 = 2$$

So weighted shortest paths need Dijkstra, assuming no negative edge weights.

24. Dijkstra vs Prim

Dijkstra and Prim can look similar because both repeatedly choose a minimum value.

But they optimize different things.

Prim:

None

Choose the cheapest edge connecting the current tree to a new vertex.

Goal: MST.

Dijkstra:

None

Choose the unvisited vertex with the smallest total distance from the source.

Goal: shortest paths from source.

Important distinction:

Prim compares edge weights.

Dijkstra compares total path distances.

Example:

Current source is A.

Known distances:

None

$$\text{dist}[B] = 4$$

```
dist[C] = 6
```

There is an edge from C to D with weight 1.

Prim might like a cheap edge of weight 1 if C is already in the tree.

Dijkstra does not choose D just because the edge is weight 1.

Dijkstra asks:

None

What is the total distance from source A to D?

Common Mistakes:

- Running Dijkstra by choosing the smallest edge.
- Running Prim by choosing the smallest total source distance.
- Forgetting that Dijkstra has a source node and distances from that source.
- Forgetting that Prim builds an MST, not shortest paths.

25. Algorithm Comparison Summary

Algorithm	Main Goal	Starts With	Repeated Choice	Output
BST insertion	Maintain BST ordering	Root	Go left/right by key comparison	Updated BST
Kruskal	Minimum spanning tree	Empty forest	Smallest safe global edge	MST
Prim	Minimum spanning tree	Start vertex	Smallest crossing edge	MST
Dijkstra	Shortest paths	Source vertex	Unvisited node with smallest distance	Shortest path tree and distances

Runtime ideas:

BST operations:

None

$O(h)$, where h is tree height

Kruskal:

None

Usually $O(E \log E)$, mostly from sorting edges

Prim:

None

Depends on implementation; commonly $O(E \log V)$ with a priority queue

Dijkstra:

None

Depends on implementation; commonly $O(E \log V)$ with a priority queue

For hand tracing, focus more on the selection rule than on implementation runtime.

26. Common Review Mistakes

BST Mistakes:

- Checking only the parent when determining a valid value.
- Forgetting ancestor bounds.
- Mixing up preorder and postorder.
- Inserting a new value in the middle of the tree instead of at an empty child.
- Redrawing the whole tree incorrectly after insertion.

Kruskal Mistakes:

- Not sorting edges first.
- Adding an edge that creates a cycle.
- Forgetting that the MST has $n - 1$ edges.
- Stopping before all vertices are connected.
- Adding edge weights from skipped edges.

Prim Mistakes:

- Ignoring the starting vertex.
- Picking the globally smallest edge instead of the smallest crossing edge.
- Forgetting to update candidate edges after adding a vertex.
- Adding an edge between two vertices already inside the tree.

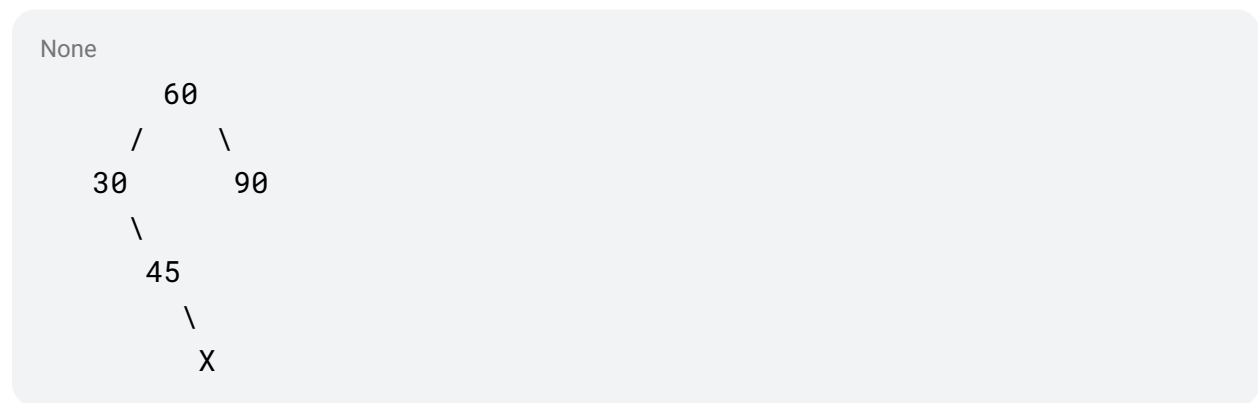
Dijkstra Mistakes:

- Treating Dijkstra like BFS.
- Choosing the smallest edge instead of the smallest total distance.
- Forgetting to update previous nodes during relaxation.
- Drawing all relaxed edges instead of the final shortest path tree.
- Forgetting to reverse previous pointers when writing the final path.
- Using Dijkstra on graphs with negative edge weights.

27. Similar Practice Problems

Problem 1: BST Bounds

Given this BST:



Give one possible value for X.

Answer:

X must be greater than 45 and less than 60.

One possible value:

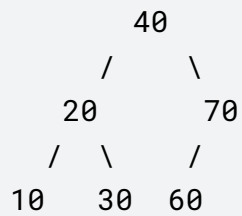
None

50

Problem 2: Preorder and Postorder

Given this tree:

None



Preorder:

None

40 20 10 30 70 60

Postorder:

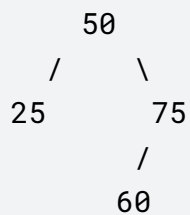
None

10 30 20 60 70 40

Problem 3: BST Insertion

Insert 65 into this BST:

None



Steps:

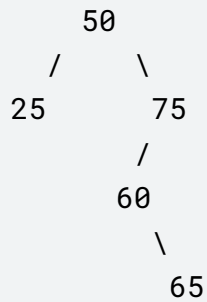
None

65 > 50, go right
65 < 75, go left

65 > 60, go right
insert 65 as right child of 60

Result:

None



Problem 4: Kruskal MST

Edges:

None

A-B: 1
A-C: 4
B-C: 2
B-D: 5
C-D: 3

Sorted edges:

None

A-B: 1
B-C: 2
C-D: 3
A-C: 4
B-D: 5

Kruskal selected edges:

None

A-B

B-C

C-D

Total cost:

None

$$1 + 2 + 3 = 6$$

Problem 5: Prim MST

Use the same graph:

None

A-B: 1

A-C: 4

B-C: 2

B-D: 5

C-D: 3

Start at A.

Step 1:

Candidate edges:

None

A-B: 1

A-C: 4

Pick A-B.

Step 2:

Tree has A, B.

Candidate edges:

None

A-C: 4

B-C: 2

B-D: 5

Pick B-C.

Step 3:

Tree has A, B, C.

Candidate edges:

None

B-D: 5

C-D: 3

Pick C-D.

MST:

None

A-B

B-C

C-D

Total cost:

None

6

Problem 6: Dijkstra

Edges:

None

A-B: 2

A-C: 6

B-C: 1
B-D: 3
C-D: 1

Start at A.

Initial:

None
A = 0
B = ∞
C = ∞
D = ∞

Process A:

None
B = 2, prev[B] = A
C = 6, prev[C] = A

Process B:

None
C through B = $2 + 1 = 3$
Update C = 3, prev[C] = B

D through B = $2 + 3 = 5$
Update D = 5, prev[D] = B

Process C:

None
D through C = $3 + 1 = 4$
Update D = 4, prev[D] = C

Final distances:

None

A = 0

B = 2

C = 3

D = 4

Shortest path from A to D:

None

A -> B -> C -> D

Cost:

None

4

28. Final Checklist

Before working on graph and tree problems, make sure you can:

- Use ancestor bounds to determine valid BST values.
- Trace preorder and postorder without skipping nodes.
- Insert into a BST and redraw only the changed branch.
- Explain what a spanning tree is.
- Know that an MST with n vertices has $n - 1$ edges.
- Run Kruskal by sorting edges and skipping cycles.
- Run Prim by growing from a start node using crossing edges.
- Apply tie-breaking rules only after the main priority rule.
- Set up a Dijkstra distance table.
- Relax edges correctly.
- Track previous nodes.
- Draw a shortest path tree from previous pointers.
- Recover a path by following previous pointers backward and reversing the result.
- Explain the difference between MST problems and shortest path problems.